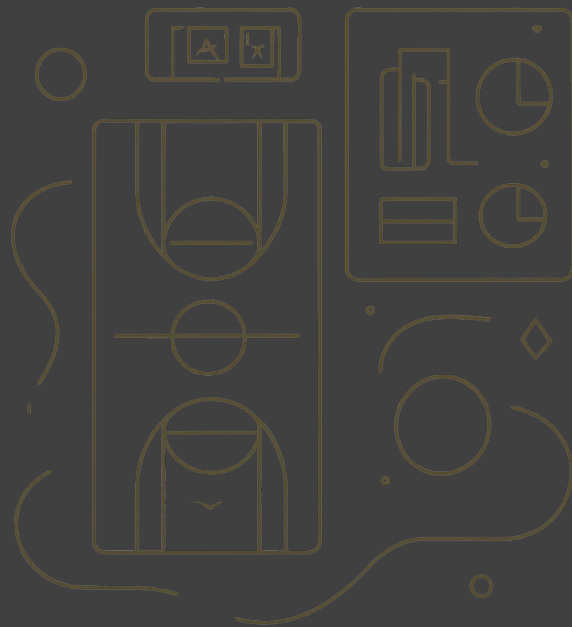


NBA HOME WINS ANALYSIS

Análisis de Partidos de la NBA con
Machine y Deep Learning

Proyecto de Inteligencia Artificial y Ciencia de Datos

Integrantes: Miguel Ángel Angulo · Juan David Lizcano · Diego Alexander
Niño



Introducción

Este proyecto analiza estadísticas de partidos de la NBA aplicando técnicas avanzadas de análisis de datos, machine learning y deep learning. Todo el trabajo fue desarrollado en **Google Colab** usando Python.

Limpieza de datos

Visualización

Modelado predictivo

Clustering

Reducción de dimensionalidad

Dataset Utilizado

Cada fila del dataset representa un **partido de la NBA**. El objetivo es encontrar patrones ocultos y realizar predicciones sobre los resultados.

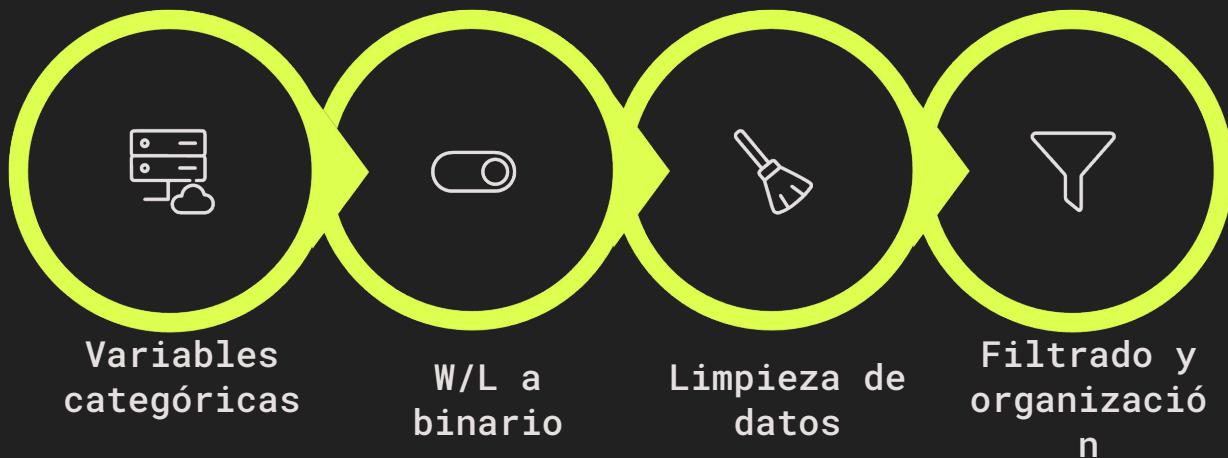
```
1 # Se lee el archivo Excel llamado "game.xlsx" y se almacena en el DataFrame "data"
2 data = pd.read_excel('game.xlsx')
3 data
```

...	team_home	team_away	game_date	wl_home	wl_away	fgm_home	ftm_home	pts_home	plus_minus_home	fgm_away	ftm_away	pts_away	season_type
0	HUS	NYK	1946-11-01	L	W	25.0	16.0	66.0	-2	24.0	20.0	68.0	Regular Season
1	BOM	PIT	1946-11-02	W	L	20.0	16.0	56.0	5	16.0	19.0	51.0	Regular Season
2	PRO	BOS	1946-11-02	W	L	21.0	17.0	59.0	6	21.0	11.0	53.0	Regular Season
3	CHS	NYK	1946-11-02	W	L	21.0	21.0	63.0	16	16.0	15.0	47.0	Regular Season
4	DEF	WAS	1946-11-02	L	W	10.0	13.0	33.0	-17	18.0	14.0	50.0	Regular Season
...	...	...	...	...	...	...	...	...	...	...	...	...	...
65693	MIA	DEN	2023-06-07	L	W	34.0	15.0	94.0	-15	41.0	22.0	109.0	Playoffs
65694	MIA	DEN	2023-06-09	L	W	35.0	17.0	95.0	-13	39.0	16.0	108.0	Playoffs
65695	DEN	MIA	2023-06-12	W	L	38.0	13.0	94.0	5	33.0	14.0	89.0	Playoffs
65696	LBN	GNS	2023-02-19	L	W	79.0	0.0	175.0	-9	76.0	3.0	184.0	All-Star
65697	LBN	GNS	2023-02-19	L	W	79.0	0.0	175.0	-9	76.0	3.0	184.0	All-Star

65698 rows × 13 columns

Preprocesamiento de Datos

El preprocesamiento es fundamental para garantizar la calidad del dataset antes de aplicar cualquier modelo de machine learning.



Estos cuatro pasos aseguran que el dataset esté correctamente estructurado, sin valores inconsistentes y listo para ser consumido por los algoritmos de modelado predictivo.

Ingeniería de Características

Nueva variable creada

Se generó la variable `total_tiros_libre`, que representa la **suma de tiros libres de ambos equipos** en cada partido.

Variables originales

Tiros libres local + tiros libres visitante por separado

Variable derivada

```
total_tiros_libre = FTM_home + FTM_away
```

Nueva variable creada

También se creó la variable **total_puntos**, correspondiente a la suma total de puntos obtenidos entre el equipo local y visitante.

Variables originales

puntos local + puntos visitante

Variable derivada

```
total_puntos = pts_home + pts_away
```

Después del preprocesamiento, el dataset quedó compuesto únicamente por variables relevantes para el análisis predictivo.

```
1 # Muestra las últimas 10 filas del DataFrame ordenado
2 d["total_tiros_libre"] = d["ftm_home"] + d["ftm_away"]
3
4 # Ordena el DataFrame de mayor a menor según el total de tiros libres
5 d = d.sort_values(by="total_tiros_libre", ascending=False)
6
7 # Muestra las primeras 5 filas con los partidos de mayor cantidad de tiros libres
8 d.head(5)
```

	team_home	team_away	game_date	wl_home	wl_away	fgm_home	ftm_home	pts_home	plus_minus_home	fgm_away	ftm_away	pts_away	season_type	total_puntos	total_tiros_libre
1041	SYR	AND	1949-11-24	1	0	33.0	59.0	125.0	2	33.0	57.0	123.0	0	248.0	116.0
2632	BOS	SYR	1953-03-21	1	0	27.0	57.0	111.0	6	27.0	51.0	105.0	1	216.0	108.0
3040	MNL	BOS	1954-11-28	1	0	34.0	47.0	115.0	7	26.0	56.0	108.0	0	223.0	103.0
933	NYK	BAL	1949-03-26	1	0	26.0	51.0	103.0	4	26.0	47.0	99.0	1	202.0	98.0
3947	PHW	MNL	1957-11-02	1	0	35.0	48.0	118.0	10	30.0	48.0	108.0	0	226.0	96.0

Visualización de Datos

Los gráficos estadísticos permitieron explorar el comportamiento de los datos, detectar anomalías y comprender la distribución de las variables clave del dataset.



Detectar outliers

Identificación visual de valores extremos en puntos y diferencias



Comparar variables

Análisis conjunto de estadísticas ofensivas y defensivas



Analizar dispersión

Distribución y variabilidad de los datos por temporada



Observar tendencias

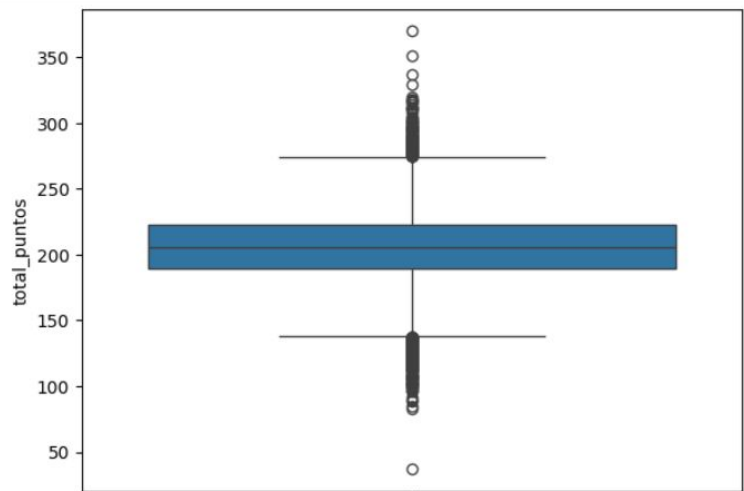
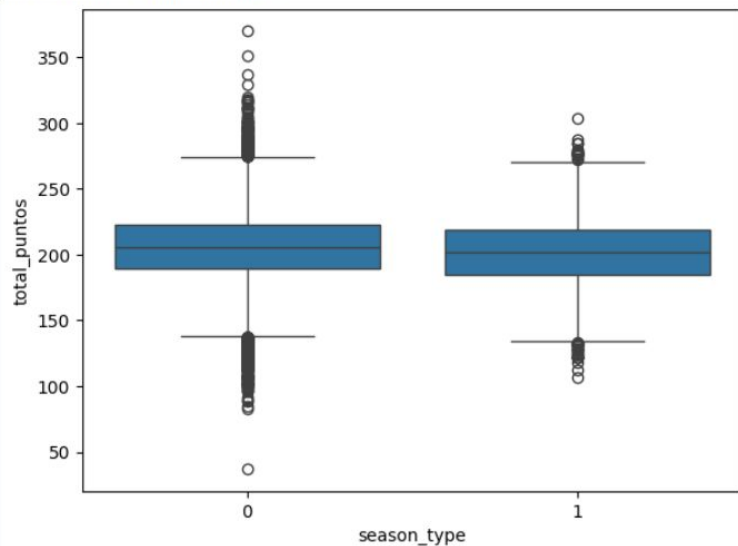
Patrones temporales y comportamientos recurrentes

Análisis Estadístico

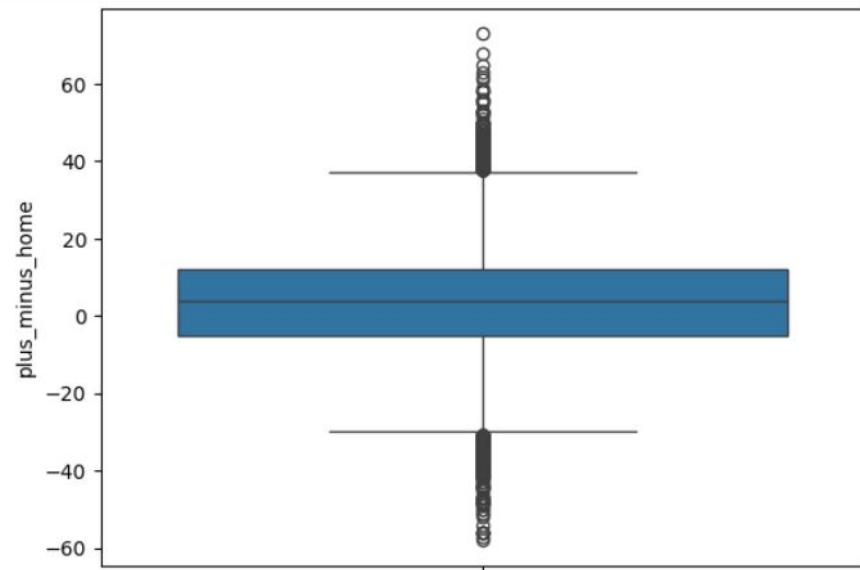
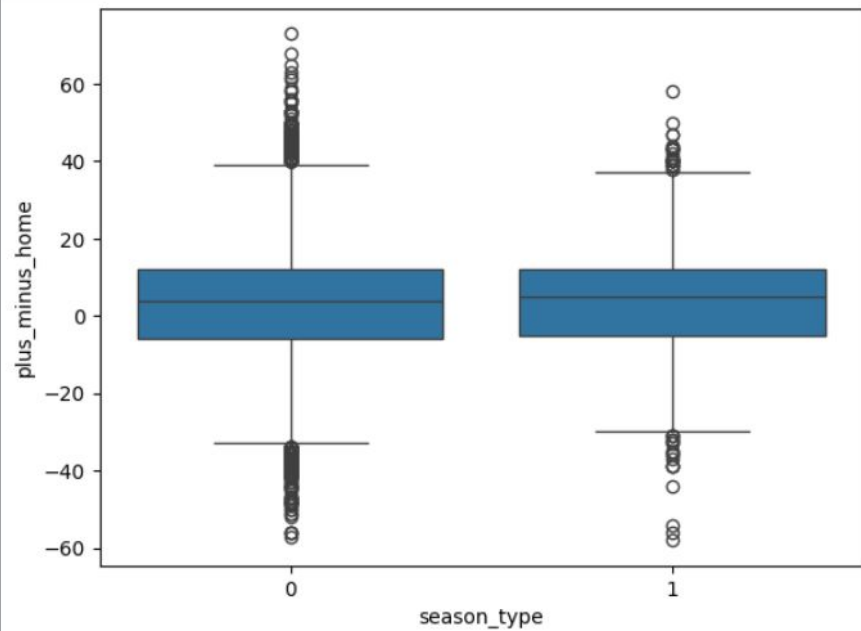
Detección de valores atípicos – Método IQR

Se aplicó el **rango intercuartílico (IQR)** para identificar partidos con comportamientos estadísticamente inusuales en las variables de puntos.

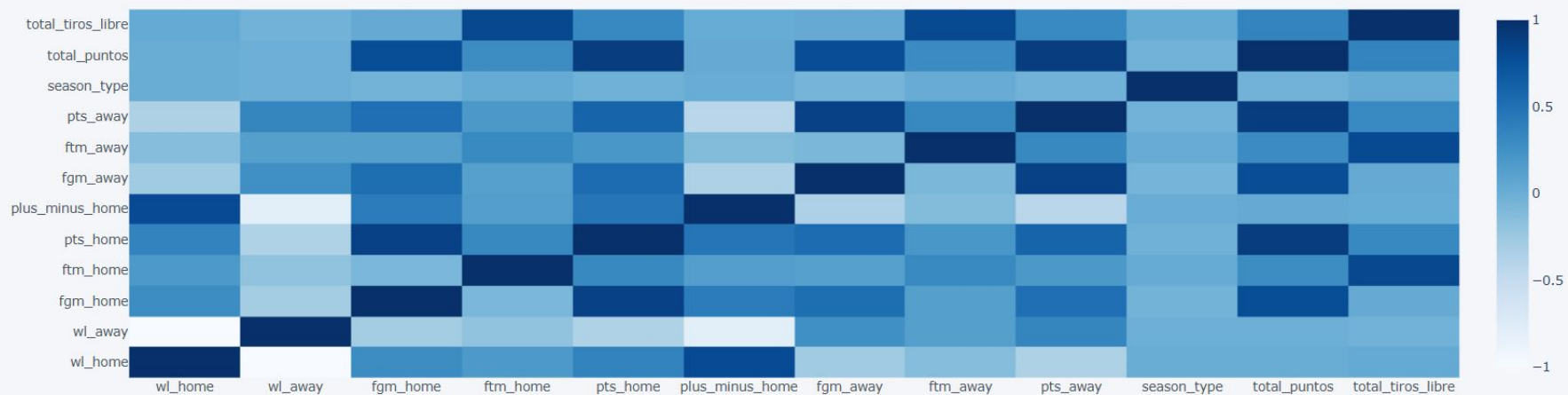
```
*** <Axes: ylabel='total_puntos'>
```



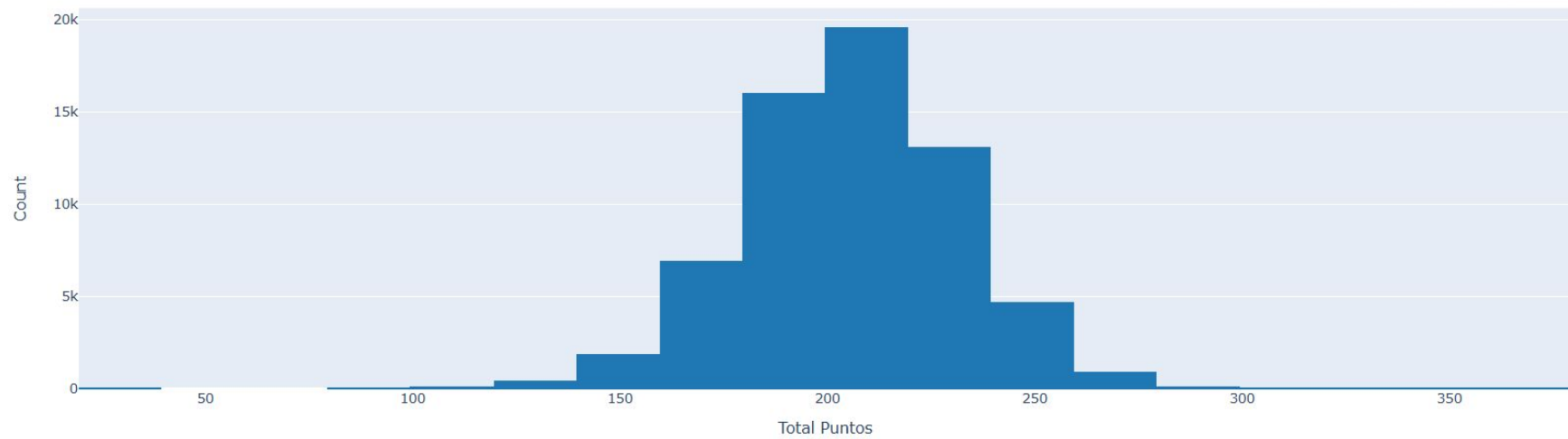
<Axes: ylabel='plus_minus_home'>



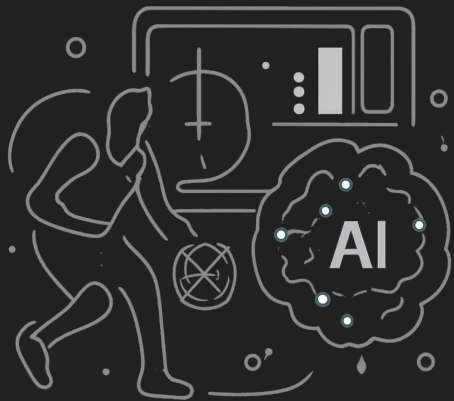
Matriz de correlación



Total puntos por game



Objetivo de Machine Learning



El proyecto busca evaluar si las **estadísticas de partido** permiten anticipar resultados de la NBA mediante modelos supervisados y no supervisados.

Problema 1

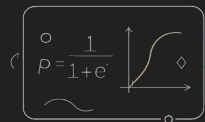
Predicción de victorias — Clasificación binaria del resultado (W/L) basada en estadísticas previas al partido

Problema 2

Clasificación de tipos de partidos — Agrupación no supervisada para identificar patrones y categorías de encuentros

Modelos Utilizados

Se implementaron y compararon distintos algoritmos de machine learning y deep learning, aplicando **validación cruzada** para garantizar la robustez de los resultados.



Regresión Logística

Modelo base de clasificación binaria para predicción de victorias



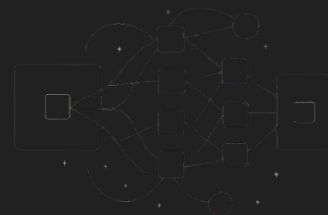
Árboles de Decisión

Modelo interpretable basado en reglas de división de variables



Random Forest

Ensamble de árboles para mejorar precisión y reducir sobreajuste



Redes Neuronales / Deep Learning

Arquitectura profunda para capturar relaciones no lineales complejas

1. Predicción de victoria por parte del local

Definición de variables y particionado

▶ 1 #@title **Definición de variables y particionado**

2 X = d.copy()

3

4 # Definimos ground truth como la variable wl_home (gana o pierde la casa)

5 y = X["wl_home"]

6

7 # Dejamos en el dataset solo las columnas de interés para la clasificación

8 X = X[["fgm_home", "ftm_home", "pts_home", "season_type"]]

9

10 # Particionamos el dataset en 80 (entrenamiento) - 20 (test)

11 # Método train_test_split contiene la permutación del dataset

12 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=21)

1 X

	fgm_home	ftm_home	pts_home	season_type
1041	33.0	59.0	125.0	0
2632	27.0	57.0	111.0	1
3040	34.0	47.0	115.0	0
933	26.0	51.0	103.0	1
3947	35.0	48.0	118.0	0
...	...	...	...	...
52907	30.0	8.0	76.0	0
14111	62.0	2.0	126.0	0
58111	40.0	6.0	96.0	0
51028	33.0	7.0	77.0	0
9261	42.0	3.0	87.0	0

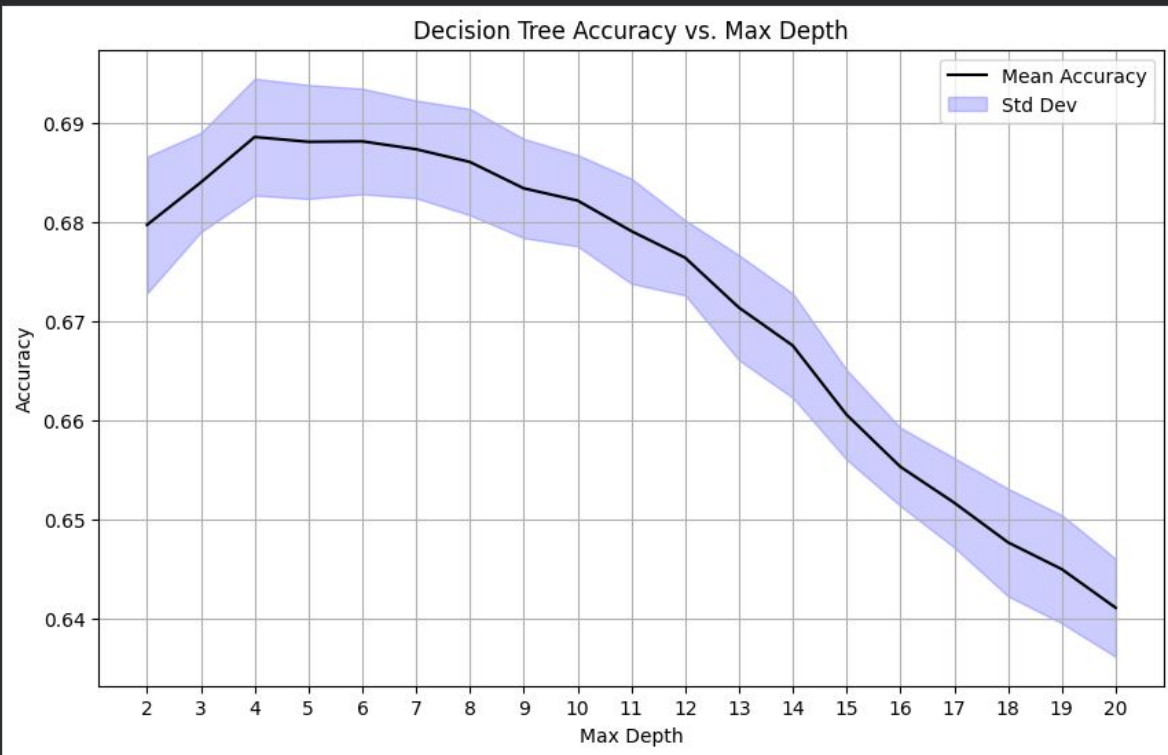
64034 rows × 4 columns

Tuning max_depth for Decision Tree



```
1 tune_DT_max_depth(X, y)
```

...



Decision Tree DT max_depth=



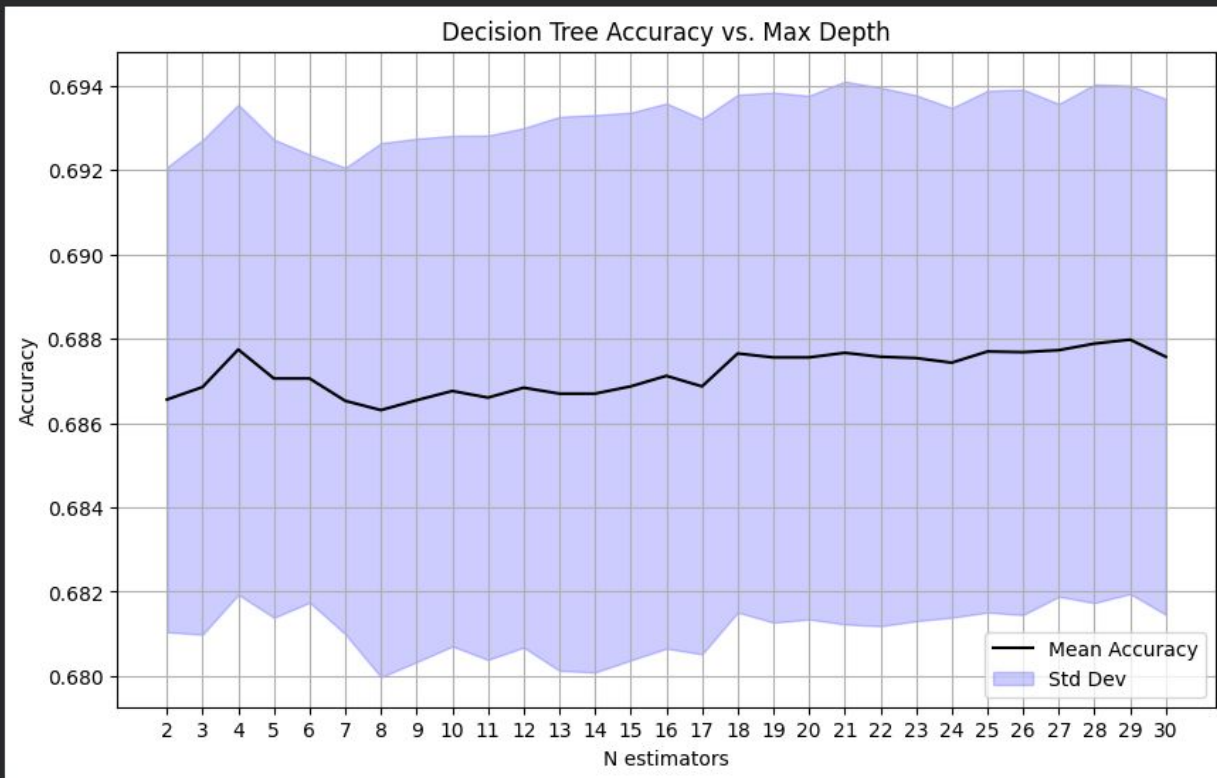
```
1 #@title **Decision Tree DT max_depth=**
2
3 # Se construye el estimador DecisionTreeClassifier
4 DT = DecisionTreeClassifier(max_depth=4, random_state=21)
5
6 # Entreno los datos DecisionTreeClassifier
7 DT.fit(X_train,y_train)
8
9 # Realizo la predicción DecisionTreeClassifier
10 y_predDT = DT.predict(X_test)
11
12 # Calculo el accuracy DecisionTreeClassifier
13 accDT = accuracy_score(y_test, y_predDT)
```

1 accDT

0.6929023190442727

Tuning max_depth for Random Forest

```
1 tune_RF_max_depth(X, y)
```



Random Forest RF SVM kernel = "linear"

```
1 #@title **Random
2
3 # Se construye el
4 RF = RandomForest
5
6 # Entreno los dat
7 RF.fit(X_train,y_
8
9 # Realizo la prec
10 y_predRF = RF.pre
11
12 # Calculo el accu
13 accRF = accuracy_
```



```
1 #@title **SVM kernel = "linear"**
2
3 # Se construye el estimador SVC
4 svc = SVC(kernel="linear", random_state=21)
5
6 # Entreno los datos SVC
7 svc.fit(X_train,y_train)
8
9 # Realizo la predicción SVC
10 y_predsvc = svc.predict(X_test)
11
12 # Calculo el accuracy SVC
13 accsvc_lin = accuracy_score(y_test, y_predsvc)
```

1 accRF

0.6940735535254158

1 accsvc_lin

0.6919653314593581

SVM KERNEL = LINEAR

SVM kernel = "linear"



```
1 #@title **SVM kernel = "linear"**  
2  
3 # Se construye el estimador SVC  
4 svc = SVC(kernel="linear", random_state=21)  
5  
6 # Entreno los datos SVC  
7 svc.fit(X_train,y_train)  
8  
9 # Realizo la predicción SVC  
10 y_predsvc = svc.predict(X_test)  
11  
12 # Calculo el accuracy SVC  
13 accsvc_lin = accuracy_score(y_test, y_predsvc)
```

```
1 accsvc_lin
```

```
0.6919653314593581
```

SVM KERNEL = POLY

SVM kernel = "poly"



```
1 #@title **SVM kernel = "poly"**
2
3 # Se construye el estimador SVC
4 svc = SVC(kernel="poly", random_state=21)
5
6 # Entreno los datos SVC
7 svc.fit(X_train,y_train)
8
9 # Realizo la predicción SVC
10 y_predsvc = svc.predict(X_test)
11
12 # Calculo el accuracy SVC
13 accsvc_pol = accuracy_score(y_test, y_predsvc)
```



```
1 accsvc_pol
```

```
0.6913406730694152
```

SVM KERNEL = RBF

SVM kernel = "rbf"



```
1 #@title **SVM kernel = "rbf"**  
2  
3 # Se construye el estimador SVC  
4 svc = SVC(kernel="rbf", random_state=21)  
5  
6 # Entreno los datos SVC  
7 svc.fit(X_train,y_train)  
8  
9 # Realizo la predicción SVC  
10 y_predsvc = svc.predict(X_test)  
11  
12 # Calculo el accuracy SVC  
13 accsvc_rbf = accuracy_score(y_test, y_predsvc)
```

```
1 accsvc_rbf
```

```
0.689544780198329
```

DeepLearning

Preparación Dataset y particionado



```
1 #@title **Preparación Dataset y particionado**
2 X2 = X.copy()
3 y2 = y.copy()
4
5 # Se calcula el número de clases con y
6 nc = len(y2.unique())
7
8 # Remove column names
9 X2.columns = range(X2.shape[1])
10 X2 = X2.to_numpy()
11 y2 = y2.to_numpy()
12 X_train2, X_test2, y_train2, y_test2 = train_test_split(X2, y2, test_size=0.2, random_state=21)
```

Primer modelo



```
1 #@title **Primer modelo**
2 # Construyo el modelo con sus capas correspondientes
3 model = tf.keras.Sequential([
4     tf.keras.layers.Flatten( input_shape= X_train2[0].shape),
5     tf.keras.layers.Dense(64, activation="relu"),
6     tf.keras.layers.Dense(128, activation="relu"),
7     tf.keras.layers.Dense(128, activation="relu"),
8     tf.keras.layers.Dense(nc, activation="softmax")]])
9
10 # Compilo el modelo con los parámetros solicitados
11 model.compile(optimizer=tf.keras.optimizers.SGD(),
12     loss='sparse_categorical_crossentropy',
13     metrics=['accuracy'])
14
15 # Entreno el modelo
16 model.fit(X_train2, y_train2, epochs=50)
17
18 # Evaluamos loss y accuracy del modelo
19 test_loss, test_acc = model.evaluate(X_test2, y_test2)
```


Segundo modelo



```
1 #@title **Segundo modelo**
2 # Construyo el modelo con sus capas correspondientes
3 model2 = tf.keras.Sequential([
4     tf.keras.layers.Flatten(input_shape= X_train2[0].shape),
5     tf.keras.layers.Dense(64, activation="relu"),
6     tf.keras.layers.Dense(64, activation="relu"),
7     tf.keras.layers.Dense(128, activation="relu"),
8     tf.keras.layers.Dense(128, activation="relu"),
9     tf.keras.layers.Dense(256, activation="relu"),
10    tf.keras.layers.Dense(256, activation="relu"),
11    tf.keras.layers.Dense(nc, activation="softmax")])
12
13 # Compilo el modelo con los parámetros solicitados
14 model2.compile(optimizer=tf.keras.optimizers.SGD(),
15     loss='sparse_categorical_crossentropy',
16     metrics=['accuracy'])
17
18 # Entreno el modelo
19 model2.fit(X_train2, y_train2, epochs=50)
20
21 # Evaluamos loss y accuracy del modelo
22 test_loss2, test_acc2 = model2.evaluate(X_test2, y_test2)
```

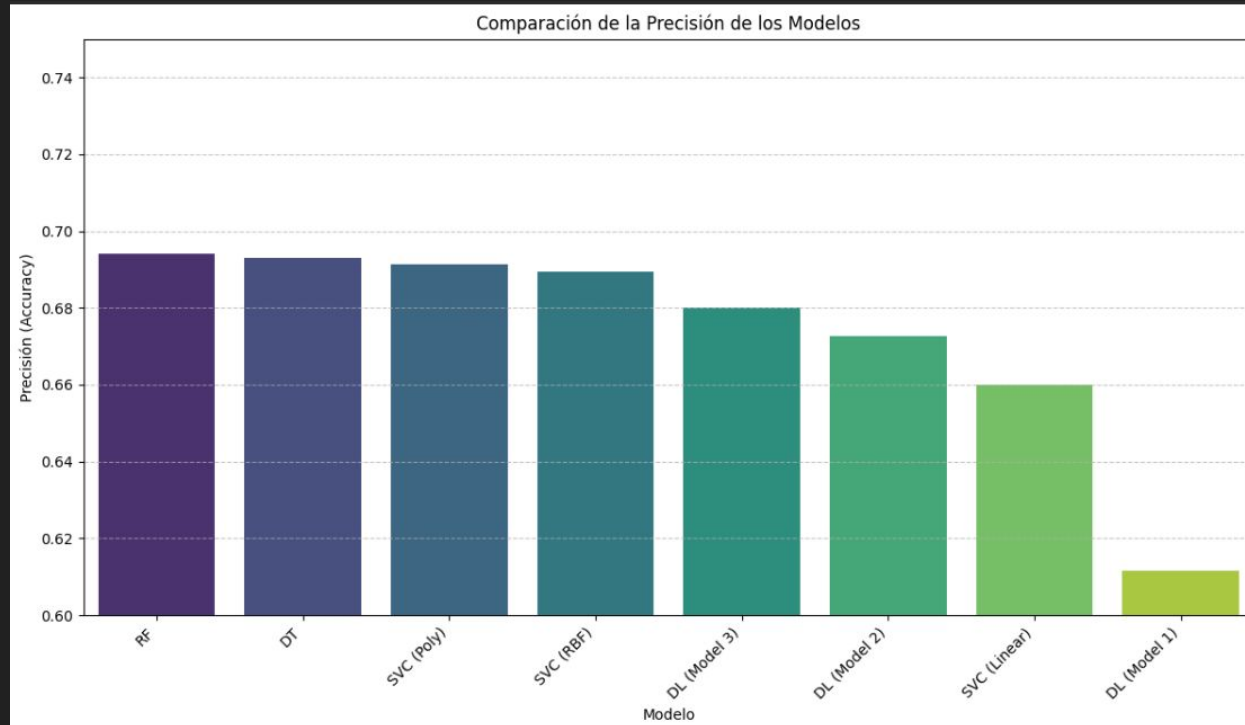
Tercer modelo



```
1 #@title **Tercer modelo**
2 # Construyo el modelo con sus capas correspondientes
3 model3 = tf.keras.Sequential([
4     tf.keras.layers.Flatten(input_shape= X_train2[0].shape),
5     tf.keras.layers.Dense(128, activation="relu"),
6     tf.keras.layers.Dense(128, activation="relu"),
7     tf.keras.layers.Dense(128, activation="relu"),
8     tf.keras.layers.Dense(128, activation="relu"),
9     tf.keras.layers.Dense(128, activation="relu"),
10    tf.keras.layers.Dense(128, activation="relu"),
11    tf.keras.layers.Dense(128, activation="relu"),
12    tf.keras.layers.Dense(128, activation="relu"),
13    tf.keras.layers.Dense(128, activation="relu"),
14    tf.keras.layers.Dense(128, activation="relu"),
15    tf.keras.layers.Dense(nc, activation="softmax")])
16
17 # Compilo el modelo con los parámetros solicitados
18 model3.compile(optimizer=tf.keras.optimizers.SGD(),
19    loss='sparse_categorical_crossentropy',
20    metrics=['accuracy'])
21
22 # Entreno el modelo
23 model3.fit(X_train2, y_train2, epochs=50)
24
25 # Evaluamos loss y accuracy del modelo
26 test_loss3, test_acc3 = model3.evaluate(X_test2, y_test2)
```

Resultados de Predicción

Los modelos fueron evaluados utilizando métricas de rendimiento. Se compararon precisión, exactitud, desempeño y capacidad de generalización. Los modelos de Deep Learning mostraron resultados competitivos frente a modelos tradicionales.



1. Predicción tipo de partido

Definición de variables y particionado

```
1 #@title **Definición de variables y particionado**
2 X = d.copy()
3
4 # Definimos ground truth como la variable wl_home (gana o pierde la casa)
5 y = X["season_type"]
6
7 # Dejamos en el dataset solo las columnas de interés para la clasificación
8 X = X[["fgm_home", "ftm_home", "pts_home", "fgm_away", "ftm_away", "pts_away"]]
9
10 # Particionamos el dataset en 80 (entrenamiento) - 20 (test)
11 # Método train_test_split contiene la permutación del dataset
12 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=21)
```



1 X

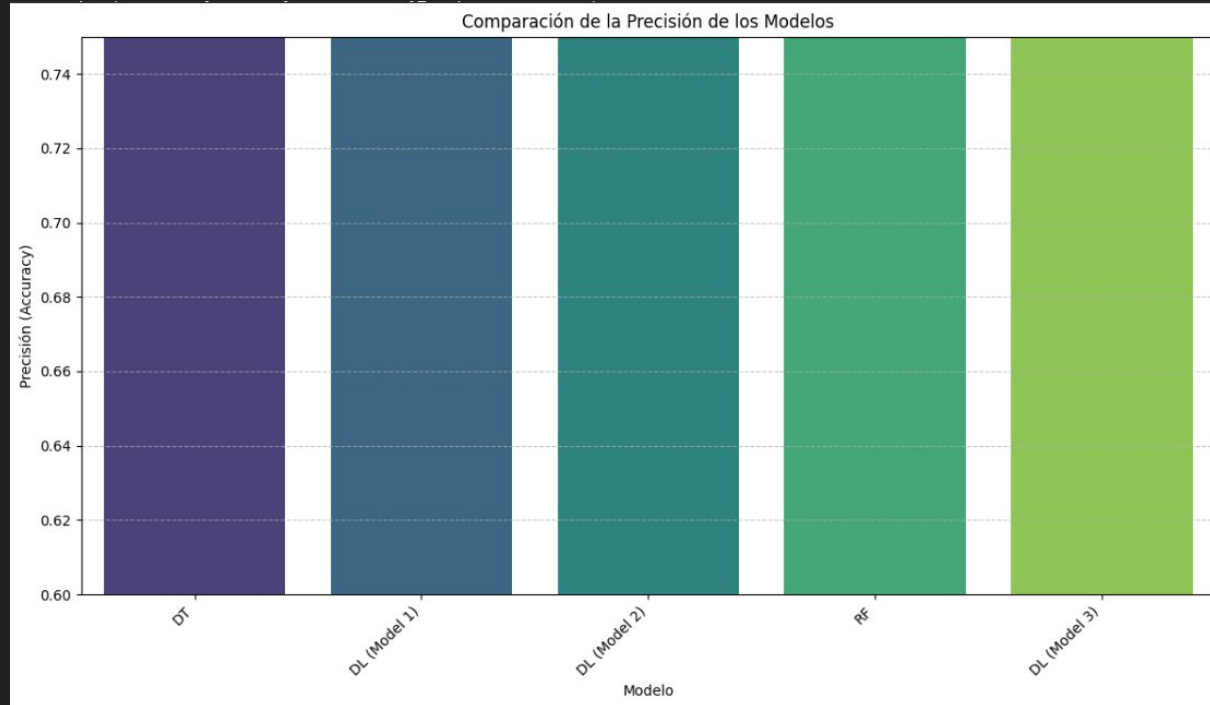
...

	fgm_home	ftm_home	pts_home	fgm_away	ftm_away	pts_away
1041	33.0	59.0	125.0	33.0	57.0	123.0
2632	27.0	57.0	111.0	27.0	51.0	105.0
3040	34.0	47.0	115.0	26.0	56.0	108.0
933	26.0	51.0	103.0	26.0	47.0	99.0
3947	35.0	48.0	118.0	30.0	48.0	108.0
...	...	...	...	...	...	...
52907	30.0	8.0	76.0	37.0	1.0	79.0
14111	62.0	2.0	126.0	40.0	7.0	88.0
58111	40.0	6.0	96.0	37.0	3.0	85.0
51028	33.0	7.0	77.0	30.0	1.0	64.0
9261	42.0	3.0	87.0	42.0	4.0	88.0

64034 rows × 6 columns

Resultados de Predicción

Los modelos fueron evaluados utilizando métricas de rendimiento. Se compararon precisión, exactitud, desempeño y capacidad de generalización. Los modelos de Deep Learning mostraron resultados competitivos frente a modelos tradicionales.

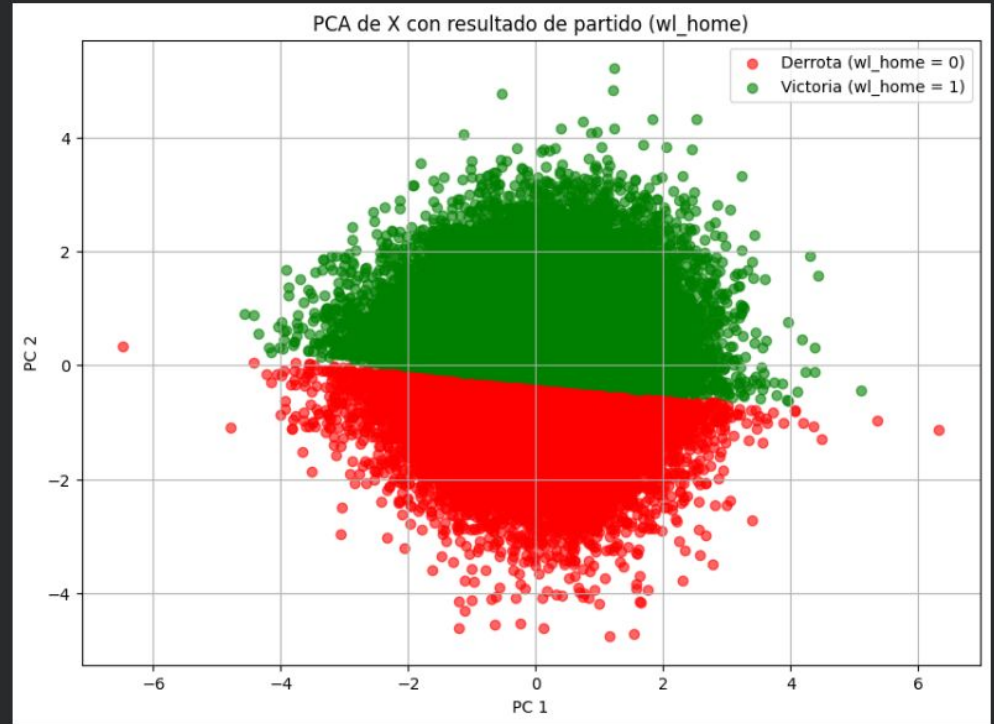


PCA – Análisis de Componentes Principales

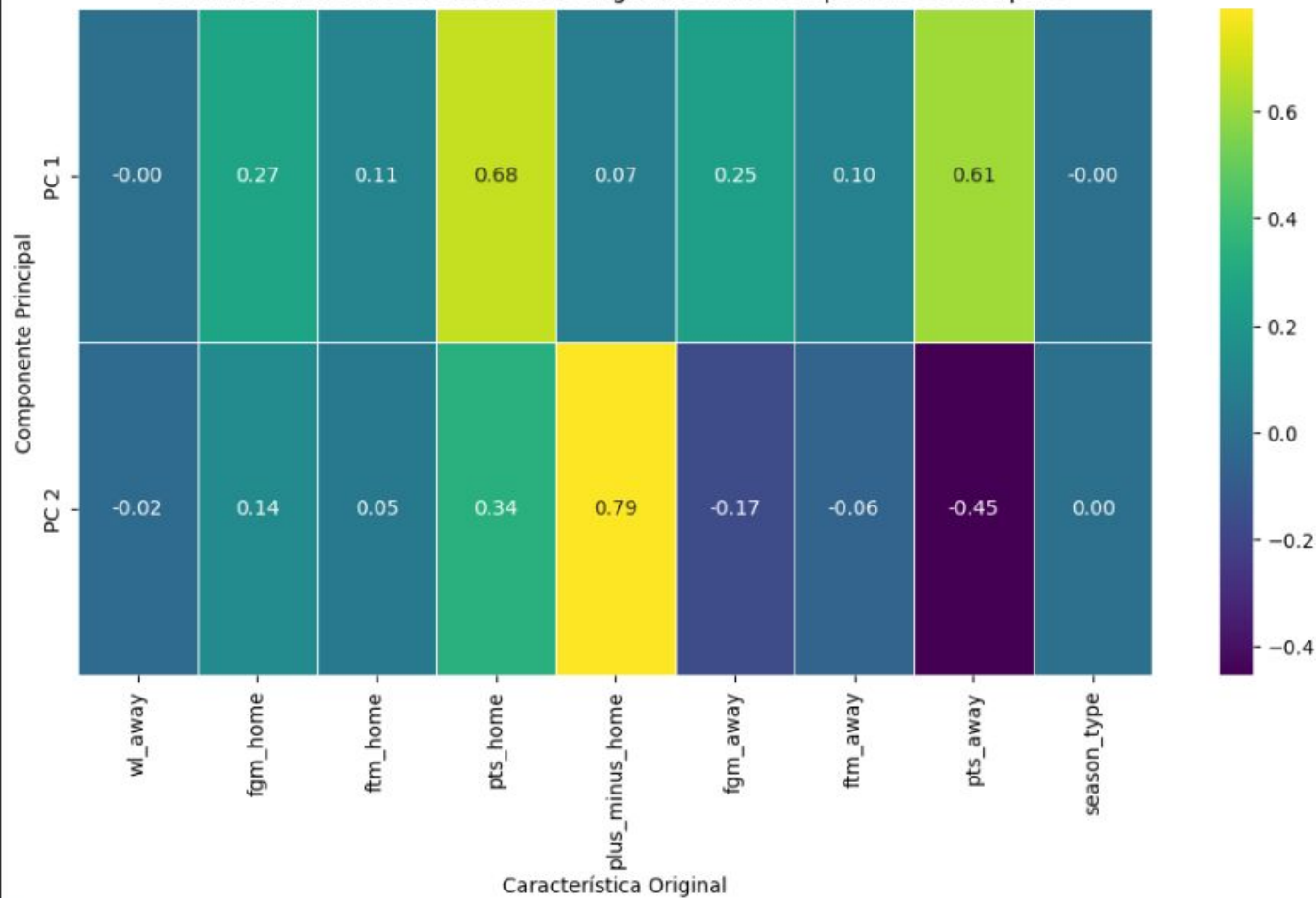
Objetivos del PCA

- Reducir la dimensionalidad del dataset
- Simplificar la visualización de datos
- Identificar patrones importantes
- Mejorar el análisis de agrupamiento

También se analizó la contribución de cada variable a las componentes principales.

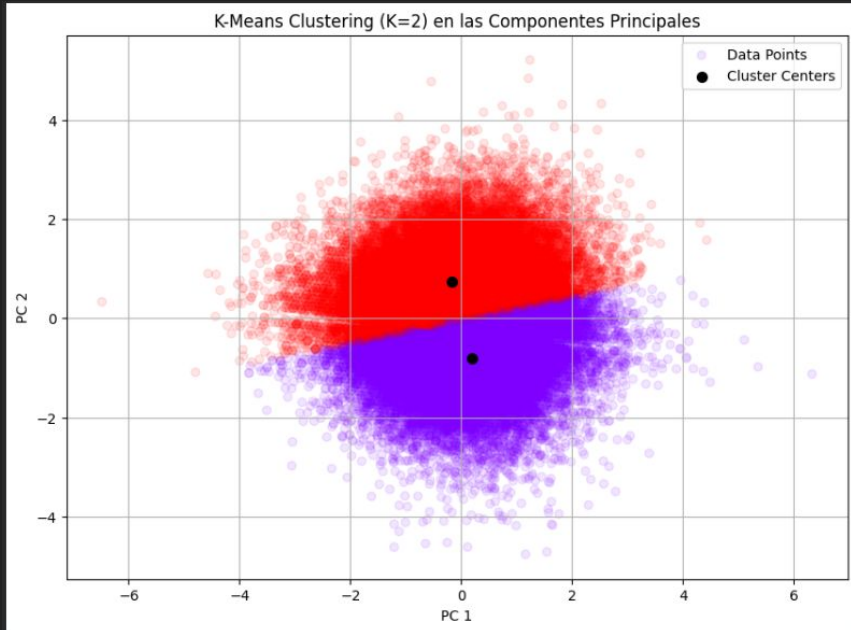


Contribución de las características originales a las Componentes Principales

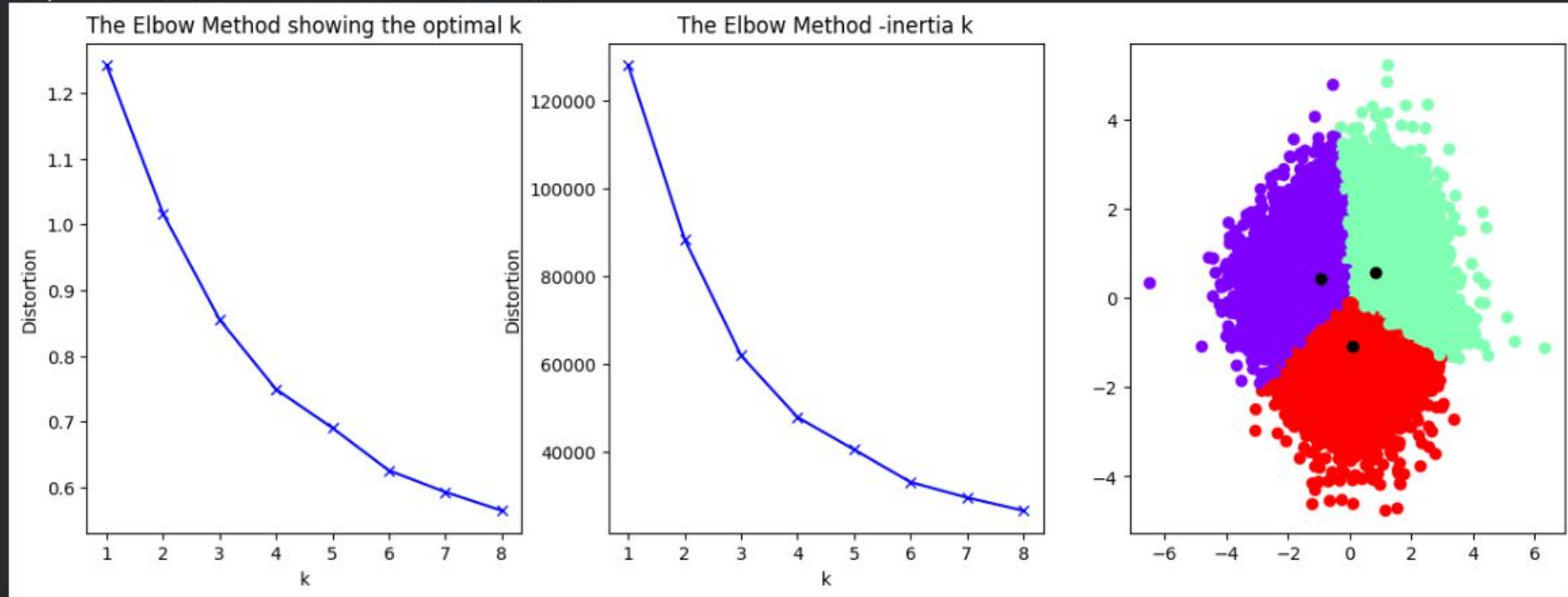


K-Means – Agrupamiento de Partidos

Se utilizó el algoritmo K-Means para agrupar partidos similares según sus estadísticas. El modelo permitió identificar grupos de partidos, detectar patrones y visualizar agrupaciones usando PCA.

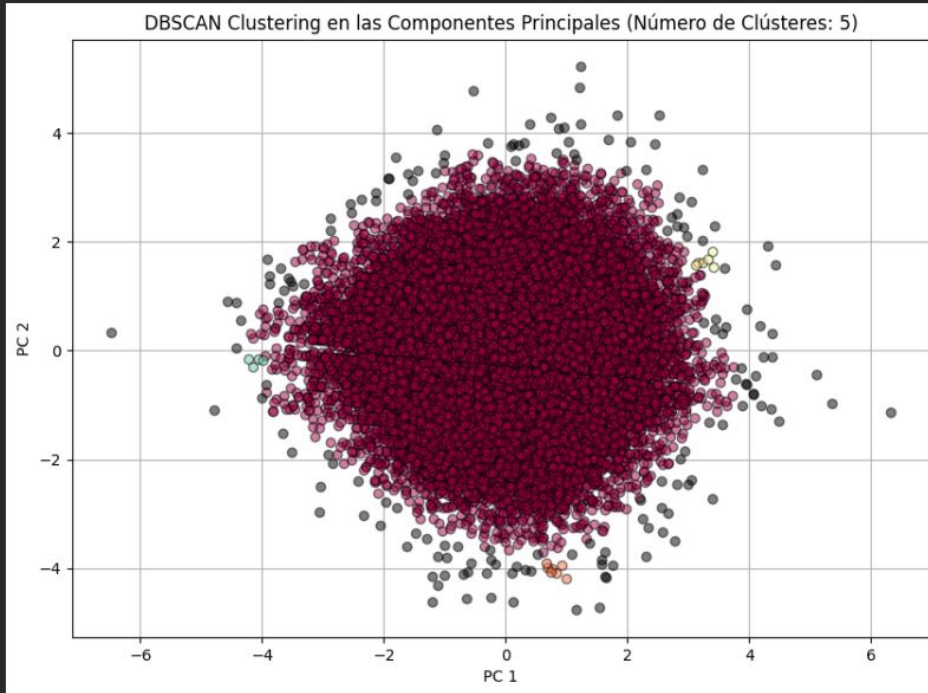


<matplotlib.collections.PathCollection at 0x7ecfc1be0980>



Se aplicó el método *Elbow* con el objetivo de determinar el número óptimo de clusters para el algoritmo K-Means.

DBSCAN – Clustering Basado en Densidad



Número estimado de clústeres: 5
Número de puntos de ruido: 163

Ventajas observadas

Detección de ruido

Identifica automáticamente puntos atípicos sin intervención manual.

Grupos no lineales

Captura estructuras complejas que K-Means no puede detectar.

Manejo de outliers

Más robusto frente a valores extremos en el dataset.

Se compararon los resultados frente a K-Means.

Conclusiones

1 Preprocesamiento clave

Fundamental para obtener buenos resultados en todos los modelos.

2 Estadísticas NBA como base

Permiten construir modelos predictivos útiles y robustos.

3 ML y Deep Learning competitivos

Ambos enfoques mostraron buen desempeño predictivo.

4 PCA simplificó el análisis

Reducción de dimensionalidad efectiva para visualización y clustering.

5 Clustering reveló patrones

K-Means y DBSCAN identificaron agrupaciones interesantes en los partidos.